AWS USER GROUP • BENGALURU • 2026

harness

Taking AI Agents to Production

AWS AgentCore × Harness AI CD

From first deployment to governed, progressive delivery.

THE PROBLEM

Building the agent is getting easier. Shipping it is not.

The prototype proves capability. Production demands control.

Identity

What can this agent access — and as whom?

Behavior

How do we know the agent is correct, not merely alive?

Release

How do we move from 0% → 10% → 50% → 100% safely?

Recovery

What is the rollback unit when behavior regresses?

Evidence

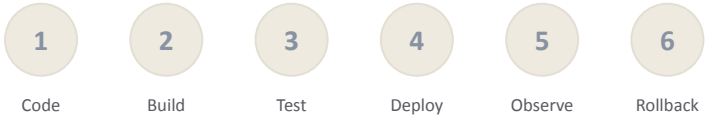
Who changed what, when, why, and with what result?

The operational problem is not "**deploy once.**"
It is "**operate releases safely over time.**"

THE PROBLEM

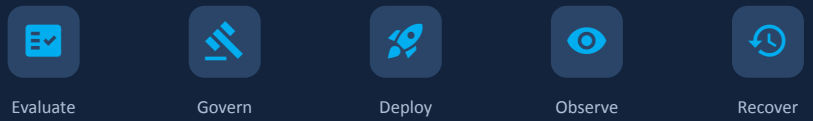
Traditional CD wasn't built for software that thinks

TRADITIONAL SERVICE



- Deploys a binary or container to a fixed target.
- Validated with deterministic unit and integration tests.
- "Healthy" means the process is up and the endpoint responds.

AI AGENT



- Deploys to an agent runtime with identity, memory, and gateway concepts.
- Binary tests confirm it runs - not that it behaves correctly.
- Needs evals: task success, safety, and tool-use correctness.

Multi-axis artifacts

Code, prompt schemas, model weights & retrieval indexes must version and roll back together.

Probabilistic evals

Quality can't be asserted with binary tests - it's measured continuously against eval datasets.

THE REAL RELEASE UNIT

An agent is not just a container.

Change any axis, and agent behavior can change.

Code
versioned source

Prompts
instructions

Model
reasoning engine

RAG / Data
context

Tools / MCP
actions

Memory
state

Policy
constraints

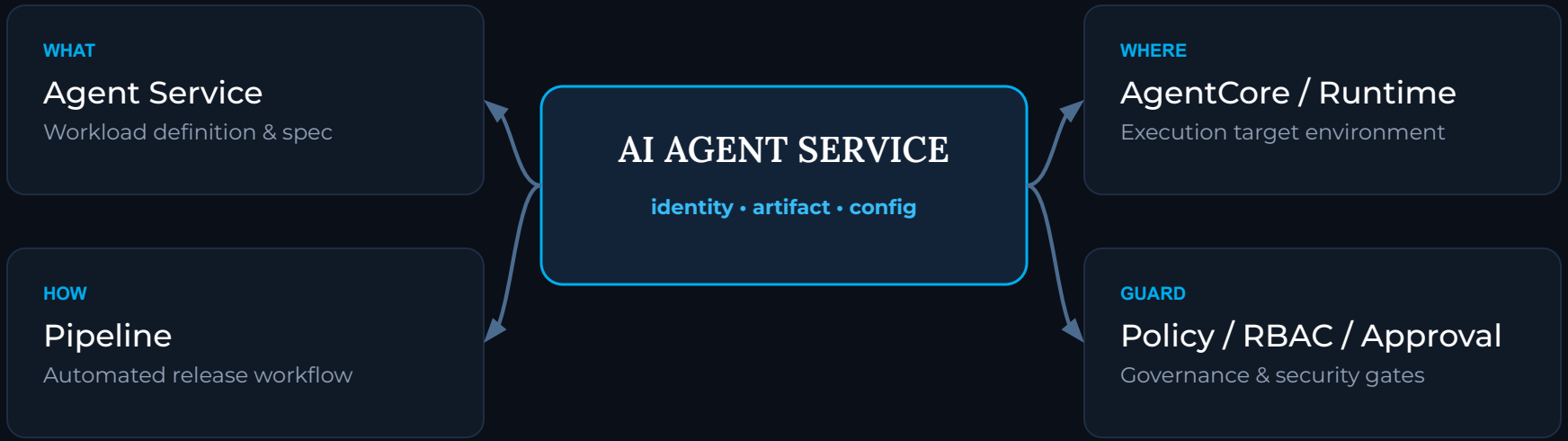
Release Unit
coordinated graph of artifacts

Think of the release as a coordinated graph of artifacts + runtime policy, not a single image tag.

THE IDEA

Make the agent a first-class CD workload.

Same Harness release model. New deployment semantics for agent runtimes.



The agent becomes something your delivery platform understands — not a script the platform happens to run.

THE ARCHITECTURE

AgentCore is the runtime. Harness is the release control plane.

They solve different layers of the production problem.

AWS RUNTIME

Amazon Bedrock AgentCore

- Managed runtime
- Sessions / execution
- Gateway
- Identity
- Observability
- Evaluations

HARNESS AI CD

Release control plane

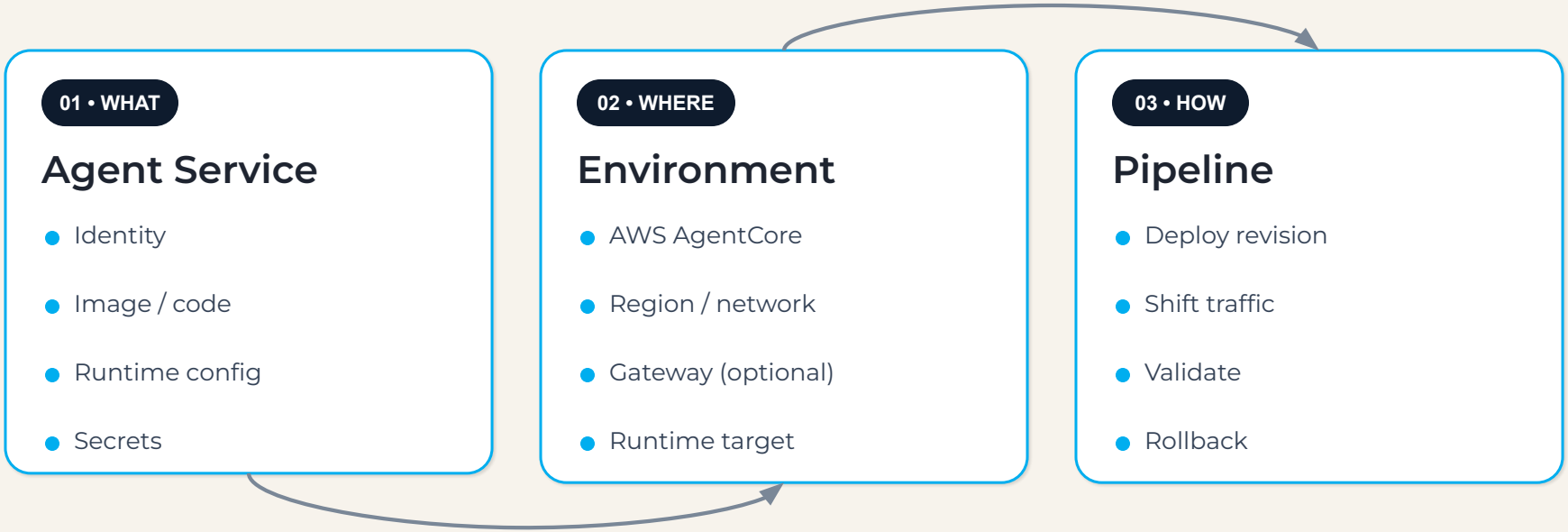
- Agent Service
- Pipeline orchestration
- Progressive traffic
- Approvals / RBAC / policy
- Audit / evidence
- Future: eval gates

Runtime executes the agent. Delivery decides when and how a revision becomes production.

THE HARNESS MODEL

Three objects. One release workflow.

WHAT am I deploying? WHERE does it run? HOW do I release it?

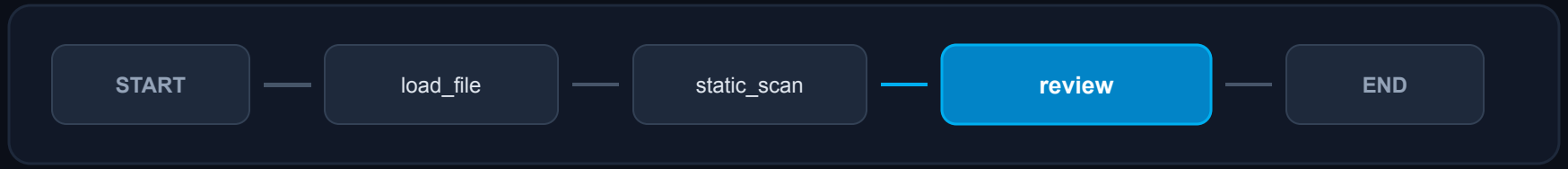


WORKFLOW FLOW: Define Workload Spec (WHAT) ➔ Target Environment (WHERE) ➔ Orchestrate Pipeline (HOW)

LIVE DEMO

Meet the agent: React Code PR Reviewer

A small LangGraph agent that turns a React file into a structured review.



Static scan

cheap heuristics produce hints

Review

LLM validates issues and returns structured JSON

Hosted

POST /invocations + GET /ping on AgentCore

Current deploy

Harness packages repo → S3 → AgentCore Runtime

DEMO ARTIFACT

React / JSX / TSX

Target code files evaluated for best practices, bugs, and performance.

LIVE DEMO

From source code to a production endpoint.

We will do the whole flow, then repeat it directly from the IDE.

01 **Define**
Agent Service

→

02 **Target**
AWS AgentCore

→

03 **Release**
Deploy + traffic

→

04 **Validate**
Smoke test

→

05 **Developer UX**
Cursor + MCP

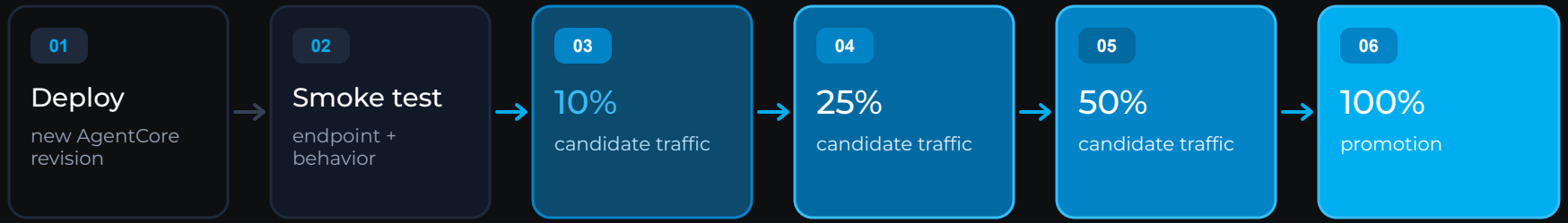
WATCH FOR THREE THINGS

1. Revision creation
2. Controlled traffic
3. Real agent behavior

DEMO / INSIDE THE PIPELINE

Deploy → validate → shift traffic → recover

The release is composed from explicit deployment primitives.



FAILURE

Rollback → previous revision

Automatic recovery path triggered if validations or smoke tests fail.

Flexible Controls

The same pipeline can insert tests, approvals or evaluations between shifts.

Key idea: traffic is mutable; the revision remains the deployment candidate.

DEMO / VALIDATION

A green deployment only proves the runtime is healthy.

So we ask the agent to review a real React file.

BuggyCounter.jsx

```jsx
const [count, setCount] = useState(0);
useEffect(() => {
 console.log(count);
}, []);

return <button>{count}</button>;
```

SMOKE TEST

ReviewReport

HOOKS	effect dependency
CORRECTNESS	stale closure
PERFORMANCE	unnecessary rerender path
MAINTAINABILITY	console.log

Infrastructure ✓ | Agent behavior ✓

DEMO / DEVELOPER EXPERIENCE

Your delivery system comes to the developer.

Deploy from Cursor / VS Code / Claude Code through MCP — without a context switch.



Developer intent

"Deploy this agent to production and run the release workflow."

NO PORTAL SWITCH

TODAY

What exists now

A first-class deployment path for managed agent runtimes — with a developer-first entry point.

DEPLOY

- AWS Bedrock AgentCore
- Google Agent Runtime
- Import existing agents
- Direct S3 code path (live)

RELEASE

- Revision-aware deployment
- Canary / traffic shifting
- Smoke tests & approvals
- Rollback to previous revision

GOVERN

- RBAC
- Policy as code
- Audit trail
- Runtime configuration + secrets

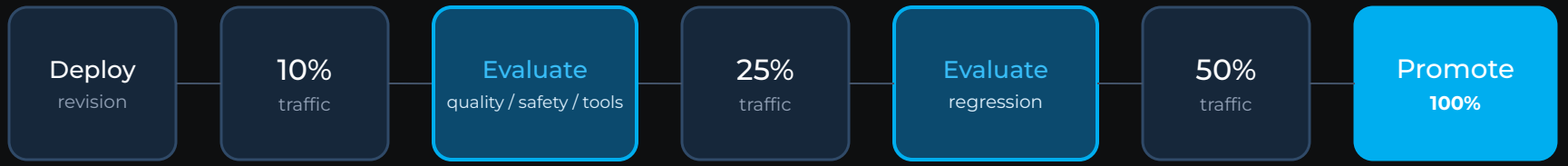
DEVELOPER UX

- Cursor / VS Code / Claude Code
- MCP interface
- No separate deployment scripts

WHAT'S NEXT

Make evaluations a native release gate.

The next question after "can it run?" is "should it receive more traffic?"



Task success
Did it achieve the goal?

Safety
Did it stay within policy?

Tool correctness
Did it call the right tool?

Cost / latency
Did the release regress ops?

"AWS AgentCore Evaluations is already a runtime primitive. Harness can turn it — and providers like Braintrust — into a release decision."

WHAT'S NEXT

Give every agent an identity — and every identity a boundary.

Treat agent permissions like a production security surface.



SECURITY ANCHOR

AGENT IDENTITY

least privilege by design

Every agent operates within explicit visual and logical security boundaries.

Allowed tools
MCP / Gateway actions

Allowed resources
databases / APIs / data

Allowed actions
read / write / execute

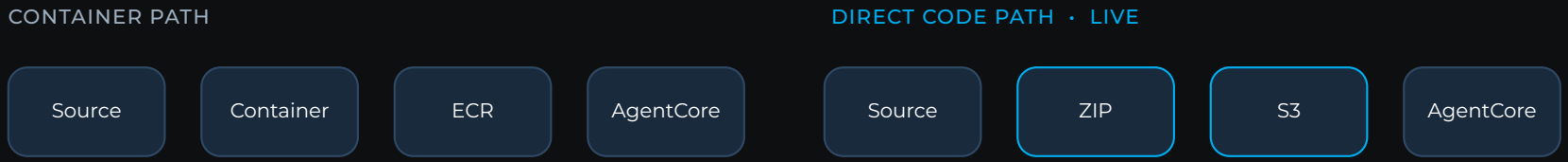
Policy
what is denied by default

Audit
who granted / changed access

TODAY + NEXT

The developer should not need Docker just to ship an agent.

Direct code deployment removes unnecessary packaging friction.



For this talk, the React review agent uses the direct S3 code path today.

LESS PLUMBING

Package what you already built. Let the platform handle the delivery workflow.

WHAT'S NEXT

Zero-touch networking.

Move Gateway provisioning and wiring out of the end-user setup checklist.

TODAY

1. Deploy first revision
2. Create AgentCore Gateway
3. Create routing rule
4. Connect Gateway to Harness
5. Then enable progressive traffic

GOAL

Create agent → deploy → release

Harness provisions and wires the Gateway needed for controlled traffic — reducing manual infrastructure work.

REMOVE SETUP TAX

WHAT'S NEXT

Four ways to make agent delivery safer and easier.

Think of the roadmap as removing a different class of production friction.

01

Evals

Smarter release decisions

- Quality
- Safety
- Tool correctness

02

Identity

Smaller blast radius

- Least privilege
- Governed access

03

Direct code

Less packaging friction

- S3 / ZIP
- Developer-first path

04

Networking

Less infra friction

- Automatic Gateway provisioning

THE GOAL

The safest production path should also be the easiest developer path.

THE BIGGER PICTURE

From deployment control to a closed-loop AI release system.

The long-term value is the feedback loop, not the deploy button.



TAKING AI AGENTS TO PRODUCTION

Thank you.

AWS AgentCore × Harness AI CD

QUESTIONS?

Let's build safer, faster AI delivery together.

The safest production path should also be the easiest developer path.